

CUSTOMIZE BEHAVIOR

Modifying system prompts

Choose between the `claude_code` preset and a custom system prompt, and customize behavior with CLAUDE.md, output styles, append, or a fully custom prompt.

System prompts define Claude's behavior, capabilities, and response style. Start from the `claude_code` preset for CLI or IDE-like coding tools where a human watches and steers the work. Write your own prompt for agents with a different surface, identity, or permission model.

This page covers:

- [How system prompts work](#), with a decision table for choosing between the preset, the preset with `append`, and a custom prompt
- [Customize agent behavior](#) with CLAUDE.md files, output styles, `append`, or a custom string
- [Compare the four approaches](#) by persistence, scope, and what they preserve
- [Combine approaches](#) to layer customization methods together

How system prompts work

A system prompt is the initial instruction set that shapes how Claude behaves throughout a conversation. The Agent SDK has three starting points for it:

- **Minimal default:** when you don't set `systemPrompt` in TypeScript or `system_prompt` in Python, the SDK uses a minimal prompt that covers tool calling but omits Claude Code's coding guidelines, response style, and project context. This differs from `claude -p`, which uses the full Claude Code prompt by default. If you're migrating from the CLI and want matching behavior, set the `claude_code` preset.
- `claude_code` **preset:** the full system prompt that the Claude Code CLI uses, with tool usage instructions, code style and formatting guidelines, response tone and verbosity rules, security and safety instructions, and context about the working directory and environment. Set `systemPrompt: { type: "preset", preset: "claude_code" }` in TypeScript or `system_prompt={"type": "preset", "preset": "claude_code"}` in Python, optionally with `append` to add your own instructions on the end.

- **Custom string:** a prompt you write yourself. The SDK sends only what you provide.

Decide on a starting point

The deciding factor is how closely your agent resembles Claude Code: a coding agent operating in a repository, with a human watching streaming output and steering the work. The further your product is from that, the more you'll want to write your own prompt.

You're building	Use	What you get
A CLI or IDE-like coding tool where a human watches and steers, and Claude Code's defaults are what you want	<code>claude_code</code> preset	The full Claude Code prompt: tool guidance, safety rules, terminal-friendly responses, repo-convention awareness
The same kind of tool, plus product-specific rules like coding standards, output format, or domain context	<code>claude_code</code> preset with <code>append</code>	Everything above, with your instructions added after the preset. Nothing is removed, so this is the lowest-risk customization
An agent with a different surface, identity, or permission model, or a non-coding agent	Custom prompt string	Only what you write. You take responsibility for replacing the tool guidance and safety instructions your agent still needs
A thin tool-calling loop with no agent persona, where you supply all behavior in the user prompt	No <code>systemPrompt</code> option	The minimal default: tool-calling support and nothing else

“Different from Claude Code” usually means one of the following:

- **Different surface:** the output isn't read in a terminal by the person who triggered it. Chat UIs, structured-output consumers, and non-coding automation each need a prompt that matches how their output is rendered and reviewed. Unattended coding automation, like a CI job that fixes lint errors or reviews diffs, still fits the preset because the work itself is what the preset is written for.
- **Different identity:** the agent shouldn't present itself as Claude Code. A support bot, a data-analysis assistant, or any domain-specific agent needs its own name, scope, and persona.
- **Different permission model:** the agent runs autonomously without a human approving each step, or operates on a narrow set of resources. Claude Code's prompt assumes a human is in the loop with access to a full toolset.
- **Non-coding tasks:** most of Claude Code's prompt is coding guidance. For research, content, or operations agents, that guidance competes with the instructions you actually need.

The [comparison table](#) shows what each customization method preserves.

Customize agent behavior

Output styles, `append`, and a custom prompt string each change the system prompt directly. CLAUDE.md takes a different path: the SDK reads it and injects its content into the conversation as project context, not into the system prompt, so it shapes behavior alongside whichever system prompt you choose. **Skills**, **hooks**, and **permissions** also shape behavior outside the system prompt and are covered on their own pages.

CLAUDE.md files for project-level instructions

CLAUDE.md files give Claude persistent project context and instructions. The SDK injects their content into the conversation, not into the system prompt, so they work with any system prompt configuration. For what to put in CLAUDE.md, where to place it, and how to write effective instructions, see **How Claude remembers your project**. This section covers what's specific to the SDK: how CLAUDE.md loads.

The SDK reads CLAUDE.md when the matching setting source is enabled: `'project'` loads `CLAUDE.md` or `.claude/CLAUDE.md` from the working directory, and `'user'` loads `~/.claude/CLAUDE.md`. Default `query()` options enable both sources, so CLAUDE.md loads automatically. If you set `settingSources` in TypeScript or `setting_sources` in Python explicitly, include the sources you need. CLAUDE.md loading is controlled by setting sources, not by the `claude_code` preset.

Load CLAUDE.md with the SDK

To load CLAUDE.md, set `settingSources` to include the level your CLAUDE.md lives at. The example below loads a project-level CLAUDE.md alongside the `claude_code` preset, so Claude has both the full coding-agent prompt and your project's conventions:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

const messages = [];

for await (const message of query({
  prompt: "Add a new React component for user profiles",
  options: {
    systemPrompt: {
      type: "preset",
      preset: "claude_code" // Use Claude Code's system prompt
    },
    settingSources: ["project"] // Loads CLAUDE.md from project
  }
})) {
  messages.push(message);
}

// Now Claude has access to your project guidelines from CLAUDE.md
```

CLAUDE.md is persistent across all sessions in a project, shared with your team through git, and discovered automatically without code changes. It is not loaded if you pass an empty `settingSources` array.

Output styles for persistent configurations

Output styles are saved configurations that modify Claude's system prompt. They're stored as markdown files and can be reused across sessions and projects.

Create an output style

An output style is a markdown file with **frontmatter** for metadata, followed by the prompt content. Save it to `~/.claude/output-styles/` for a user-level style available in every project, or `.claude/output-styles/` in your repository for a project-level style you can commit and share with your team.

By default, a custom output style replaces the `claude_code` preset's software engineering instructions with your own. To keep them and layer your instructions on top, set `keep-coding-instructions: true` in the frontmatter. Keep them when your agent is still doing software engineering work. Leave them out when you're replacing the role entirely.

The example below defines a code-review persona that keeps the coding instructions, since reviewing code still benefits from Claude Code's security and code-quality guidance. Save it as `~/.claude/output-styles/code-reviewer.md` to make it available across projects:

```
~/.claude/output-styles/code-reviewer.md
```

```
---  
name: Code Reviewer  
description: Thorough code review assistant  
keep-coding-instructions: true  
---
```

You are an expert code reviewer.

For every code submission:

1. Check for bugs and security issues
2. Evaluate performance
3. Suggest improvements
4. Rate code quality (1-10)

Activate an output style

Once created, activate output styles via:

- **CLI:** run `/config` and select an output style
- **Settings:** set `outputStyle` in `.claude/settings.local.json`
- **TypeScript SDK:** set `outputStyle` inside the inline `settings` object passed to `query()`, or point `settings` at a settings file that sets it. `outputStyle` is not a top-level `Options` field

The Python SDK does not have an option to select an output style programmatically. For code-only deployments where you can't write to `.claude/settings.local.json`, use `append` or a custom prompt string instead.

Note for SDK users: Output styles are loaded when you include `settingSources: ['user']` or `settingSources: ['project']` (TypeScript) / `setting_sources=["user"]` or `setting_sources=["project"]` (Python) in your options.

Append to the `claude_code` preset

You can use the Claude Code preset with an `append` property to add your custom instructions while preserving all built-in functionality.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

const messages = [];

for await (const message of query({
  prompt: "Help me write a Python function to calculate fibonacci numbers",
  options: {
    systemPrompt: {
      type: "preset",
      preset: "claude_code",
      append: "Always include detailed docstrings and type hints in Python
code."
    }
  }
})) {
  messages.push(message);
  if (message.type === "assistant") {
    console.log(message.message.content);
  }
}
```

Improve prompt caching across users and machines

By default, two sessions that use the same `claude_code` preset and `append` text still cannot share a prompt cache entry if they run from different working directories. This is because the preset embeds per-session context in the system prompt ahead of your `append` text: the working directory, whether it's a git repository, the platform, the active shell, the OS version, and auto-memory paths. Any difference in that context produces a different system prompt and a cache miss. CLAUDE.md content doesn't affect the system prompt cache because the SDK injects it into the conversation, not the system prompt.

To make the system prompt identical across sessions, set `excludeDynamicSections: true` in TypeScript or `"exclude_dynamic_sections": True` in Python. The per-session context moves into the first user message, leaving only the static preset and your `append` text in the system prompt so identical configurations share a cache entry across users and machines.

❗ `excludeDynamicSections` requires `@anthropic-ai/claude-agent-sdk v0.2.98` or later, or `claude-agent-sdk v0.1.58` or later for Python. It applies only to the preset object form and has no effect when `systemPrompt` is a string.

The following example pairs a shared `append` block with `excludeDynamicSections` so a fleet of

agents running from different directories can reuse the same cached system prompt:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Triage the open issues in this repo",
  options: {
    systemPrompt: {
      type: "preset",
      preset: "claude_code",
      append: "You operate Acme's internal triage workflow. Label issues by
component and severity.",
      excludeDynamicSections: true
    }
  }
})) {
  // ...
}
```

Tradeoffs: the working directory, the git-repo flag, the platform, the active shell, the OS version, and auto-memory paths still reach Claude, but as part of the first user message rather than the system prompt. Instructions in the user message carry marginally less weight than the same text in the system prompt, so Claude may rely on them less strongly when reasoning about the current directory or auto-memory paths. Enable this option when cross-session cache reuse matters more than maximally authoritative environment context.

For the equivalent flag in non-interactive CLI mode, see `--exclude-dynamic-system-prompt-sections`.

Custom system prompts

You can provide a custom string as `systemPrompt` to replace the default entirely with your own instructions.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

const customPrompt = `You are a Python coding specialist.
Follow these guidelines:
- Write clean, well-documented code
- Use type hints for all functions
- Include comprehensive docstrings
- Prefer functional programming patterns when appropriate
- Always explain your code choices`;

const messages = [];

for await (const message of query({
  prompt: "Create a data processing pipeline",
  options: {
    systemPrompt: customPrompt
  }
})) {
  messages.push(message);
  if (message.type === "assistant") {
    console.log(message.message.content);
  }
}
```

Compare the four approaches

The four customization methods differ in where they live, how they’re shared, and what they preserve from the `claude_code` preset.

Feature	CLAUDE.md	Output Styles	<code>systemPrompt</code> with <code>append</code>	Custom <code>systemPrompt</code>
Persistence	Per-project file	Saved as files	Session only	Session only
Reusability	Per-project	Across projects	Code duplication	Code duplication
Management	On filesystem	CLI + files	In code	In code
Default tools	Preserved	Preserved	Preserved	Lost (unless included)
Built-in safety	Maintained	Maintained	Maintained	Must be added
Environment context	Automatic	Automatic	Automatic	Must be provided

Feature	CLAUDE.md	Output Styles	<code>systemPrompt</code> with <code>append</code>	Custom <code>systemPrompt</code>
Customization level	Additions only	Replace or extend default	Additions only	Complete control
Version control	With project	Yes	With code	With code
Scope	Project-specific	User or project	Code session	Code session

“With append” means using `systemPrompt: { type: "preset", preset: "claude_code", append: "..."}` in TypeScript or `system_prompt={"type": "preset", "preset": "claude_code", "append": "..."}` in Python. CLAUDE.md doesn’t change the system prompt itself: the SDK injects its content into the conversation as project context.

Use cases and best practices

When to use CLAUDE.md

Use CLAUDE.md for instructions that should apply to every session in a project, regardless of which system prompt the session uses: coding standards, common commands, architecture context, and team conventions. CLAUDE.md is committed to your repository, so it stays in sync with the code it describes. See [When to add to CLAUDE.md](#) for full guidance.

CLAUDE.md files load when the `project` setting source is enabled, which it is for default `query()` options. If you set `settingSources` in TypeScript or `setting_sources` in Python explicitly, include `'project'` to keep loading project-level CLAUDE.md.

When to use output styles

Output styles are for personas you want to reuse across the CLI and SDK without changing application code. Because they live as files in `.claude/output-styles`, the same persona is available from `/config` in the CLI and from any SDK session that loads the matching setting source.

Best for:

- Persistent behavior changes across sessions
- Team-shared configurations
- Specialized assistants like a code reviewer, data scientist, or DevOps assistant
- Complex prompt modifications that need versioning

Examples:

- Creating a dedicated SQL optimization assistant
- Building a security-focused code reviewer
- Developing a teaching assistant with specific pedagogy

When to use `systemPrompt` with `append`

Use `append` when the `claude_code` preset already fits your product and you only need to layer in extra instructions. You keep the preset's tool guidance, safety rules, and coding conventions without reimplementing them.

Best for:

- Adding specific coding standards or preferences
- Customizing output formatting
- Adding domain-specific knowledge
- Modifying response verbosity
- Enhancing Claude Code's default behavior without losing tool instructions

When to use custom `systemPrompt`

Use a custom prompt when your agent's surface, identity, or permission model differs from Claude Code's, as described in [Decide on a starting point](#). You define the full instruction set, including any tool guidance and safety rules your agent needs.

Best for:

- Complete control over Claude's behavior
- Specialized single-session tasks
- Testing new prompt strategies
- Situations where default tools aren't needed
- Building specialized agents with unique behavior

Combine approaches

These methods compose. A persistent output style or CLAUDE.md sets the long-lived behavior, and `append` layers session-specific instructions on top without touching the saved configuration.

Combine an output style with session-specific additions

The example below assumes a Code Reviewer output style is already active. The `append` block layers session-specific focus areas on top of the persona, so a single review session can prioritize OAuth and token storage without changing the saved output style:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Assuming "Code Reviewer" output style is active (via /config or settings)
// Add session-specific focus areas
const messages = [];

for await (const message of query({
  prompt: "Review this authentication module",
  options: {
    systemPrompt: {
      type: "preset",
      preset: "claude_code",
      append: `
        For this review, prioritize:
        - OAuth 2.0 compliance
        - Token storage security
        - Session management
      `,
    },
  },
})) {
  messages.push(message);
}
```

See also

- **Output styles**: create, manage, and share output styles for the CLI, including the file format and storage locations
- **How Claude remembers your project**: what to put in CLAUDE.md, where to place it, and how to

write effective project instructions

- **TypeScript SDK reference**: the full `Options` type, including `systemPrompt`, `settingSources`, and `settings`
- **Python SDK reference**: the full `ClaudeAgentOptions` type, including `system_prompt` and `setting_sources`
- **Settings**: the `settings.json` reference, including where output styles and other configuration are stored